Práctica 8

CI/CD con GitHub Actions, Vercel y Render

Modalidad: pipeline guiado de integración y despliegue continuo sobre una aplicación de dos servicios, una API Spring Boot y un front-end React, con hooks de Husky en el equipo del estudiante (Parte A) + reto de diagnóstico con un repositorio y sus workflows con defectos sembrados (Parte B) + ejercicios propuestos (Parte C).

Duración estimada: 3 periodos de laboratorio.

Requisito: haber completado las Prácticas 1 a 6 (la 7, el Plan de Calidad, es el marco: el pipeline es la evidencia automática de los controles que allí se definen). Cuentas gratuitas de GitHub, Vercel y Render. Git, Node 22, Java 17 y Maven 3.9 en tu máquina.

Material: `practica8-proyectos.zip` con los proyectos `descuentos-app` (Parte A) y `reto-cicd` (Parte B).

Entregable: documento PDF con capturas, el historial de runs y la tabla de defectos corregidos + enlaces a los dos repositorios de GitHub y a la aplicación desplegada, subidos a Classroom.

1. Objetivos

Al terminar la práctica el estudiante será capaz de:

- Distinguir integración continua, entrega continua y despliegue continuo, y ubicar cada uno en el flujo que va de un commit a producción.
- Proteger cada commit en el equipo local con **Husky**: formato y análisis estático con lint-staged, mensajes con **commitlint** (Conventional Commits) y pruebas rápidas antes del push.
- Escribir workflows de **GitHub Actions** en YAML: eventos, jobs en paralelo, `needs`, matriz de versiones, caché, artefactos y workflows reutilizables (`workflow_call`).
- Automatizar en el servidor las prácticas anteriores: pruebas JUnit con cobertura mínima (Prácticas 1 y 2), análisis estático y pruebas de Vitest, y pruebas E2E con Cypress (Práctica 4).
- Desplegar una API en **Render** y un front-end en **Vercel** desde el pipeline, con un despliegue de *preview* por cada pull request y un despliegue a producción por cada merge a `main`.
- Manejar secretos, variables y ambientes de GitHub sin exponerlos en el repositorio ni en los logs.
- Verificar un despliegue con un **smoke test** que comprueba salud, versión (commit desplegado), cálculo, CORS y front-end.
- Distinguir un run verde de un pipeline sano: detectar un pipeline que no ejecuta pruebas, que ignora las que fallan, que despliega sin esperar al CI o que no recibe sus secretos.
- Diagnosticar un run fallido localizando la capa responsable (hooks locales, workflow, proyecto o código, plataforma) y corregirla con un commit por defecto.

2. Requisitos previos

Herramienta	Versión	Cómo verificar / nota
Git	2.40 o superior	`git --version`. Configura `user.name` y `user.email` antes del primer commit. En Windows incluye Git Bash, que ejecuta los hooks.

Herramienta	Versión	Cómo verificar / nota
Cuenta de GitHub	—	Los dos repositorios serán **públicos**: así los minutos de Actions no tienen costo.
Cuenta de Vercel	Plan Hobby	Regístrate con tu cuenta de GitHub en vercel.com.
Cuenta de Render	Plan Free	Regístrate con tu cuenta de GitHub en render.com.
Node.js	22 LTS o superior	`node -v`. Trae `npm`. Lo usan Husky, el front-end y Cypress.
Java (JDK)	17 o superior	`java -version`. Las clases se compilan para Java 17 aunque tengas una versión mayor.
Maven	3.9.x	`mvn -v`. El pre-push y las pruebas locales lo usan; en Actions lo trae el runner.
Navegador	Chrome o Edge	Para ver la aplicación desplegada y, en local, para Cypress (se descarga solo con `npm ci`, unos 200 MB).
Espacio en disco	2 GB	`node_modules`, el binario de Cypress y el repositorio local de Maven.
Editor	VS Code o IntelliJ	Para editar los workflows YAML, el código Java y el TypeScript.

> **Antes de la clase:** el docente debe crear su propia cuenta de prueba en Vercel y en Render y recorrer los Pasos 6 a 9 un día antes. Los planes gratuitos y sus menús cambian con el tiempo; si algo no coincide con la guía, el Anexo C resume qué se necesita de cada plataforma. El plan gratuito de Render duerme el servicio tras 15 minutos sin tráfico y tarda alrededor de un minuto en despertar: el smoke test ya espera esa demora.

3. CI/CD: qué es y dónde encaja

En la Práctica 6 Jenkins ejecutaba, en una máquina neutral, los comandos de las prácticas anteriores cada vez que llegaba un commit. Eso es **integración continua**. Esta práctica la repite con otra herramienta y da el paso que faltaba: lo que pasó las pruebas **llega solo a producción**, y después se comprueba que llegó bien.

Término	Qué significa	En esta práctica
Integración continua (CI)	Cada cambio se compila y se prueba automáticamente en un entorno limpio.	El workflow `ci.yml`: API, front-end y E2E en cada pull request y en cada push a `main`.
Entrega continua	Todo cambio que pasa el CI queda listo para desplegarse; una persona decide cuándo.	El preview por pull request. Con una aprobación manual en `web-production` (Ejercicio 3) el despliegue a producción también sería entrega continua.
Despliegue continuo (CD)	Todo cambio que pasa el CI se despliega a producción sin intervención.	El workflow `cd.yml`: un merge a `main` despliega la API y el front-end y ejecuta el smoke test.

Jenkins y GitHub Actions

Los conceptos son los mismos: disparo, etapas, resultado, artefactos, secretos. Cambia dónde corre cada cosa y cómo se escribe.

Aspecto	Jenkins (Práctica 6)	GitHub Actions (Práctica 8)
Dónde corre	En un contenedor de tu máquina.	En máquinas virtuales de GitHub (runners) que nacen limpias y se destruyen al terminar.
Quién lo instala	Tú: Docker, plugins, herramientas.	Nadie: viene con el repositorio.
Pipeline	`Jenkinsfile` (Groovy), en la raíz.	Archivos YAML en `.github/workflows/`.
Disparo	`pollSCM` cada 2 minutos: Jenkins pregunta.	Eventos (`push`, `pull_request`) al instante: GitHub avisa.
Reutilizar pasos	Plugins y librerías compartidas.	Actions del Marketplace (`uses:`) y workflows reutilizables (`workflow_call`).
Secretos	Credentials de Jenkins.	Secrets, Variables y Environments del repositorio.
Paralelismo	Etapas en secuencia (`stages`).	Jobs en paralelo, ordenados con `needs`.
Resultado	Build #N con Stage View.	Run #N con el grafo de jobs, un resumen y artefactos.

Cómo encaja con las prácticas anteriores

Práctica	Qué se automatizó	Comando	Dónde queda en el pipeline
1 y 2 — JUnit y TDD	Pruebas unitarias de la API	`mvn verify`	Job `API`, con cobertura mínima de 80 % (JaCoCo)
3 — SonarQube	Análisis estático y Quality Gate	`sonar:sonar`	ESLint en el front-end; SonarCloud en el Ejercicio 5
4 — Cypress	Pruebas E2E en navegador	`cypress run`	Job `E2E`, contra la API y el front-end reales
5 — JMeter	Pruebas de rendimiento contra un SLA	`jmeter -n`	Opcional en el Ejercicio 5, contra producción
6 — Jenkins	Orquestar todo en cada cambio	`Jenkinsfile`	Aquí se hace con GitHub Actions
7 — Plan de Calidad	Definir qué controles exige el proyecto	SQAP	Cada job del pipeline es la evidencia automática de un control del plan

Tres barreras, tres lugares

Barrera	Dónde corre	Qué detiene	¿Se puede saltar?
Husky: `pre-commit`, `commit-msg`, `pre-push`	Tu máquina	Código sin formato o con errores de lint, mensajes de commit mal escritos, pruebas rápidas en rojo.	Sí: `git commit --no-verify`.
CI en GitHub Actions	Servidor neutral	Todo lo anterior, más la matriz de versiones, la cobertura y el E2E.	No, si la rama `main` está protegida (Ejercicio 2).

Barrera	Dónde corre	Qué detiene	¿Se puede saltar?
Smoke test posterior al despliegue	Producción	Un despliegue roto, la versión equivocada, una API que el front-end no puede llamar (CORS).	No: el run queda en rojo.

Los hooks de Husky son una **comodidad**: te avisan en segundos, antes de subir. La **garantía** es el servidor, porque nadie puede saltarse lo que corre en él. Por eso el job `Mensajes de commit` repite en GitHub la validación que `commit-msg` hace en tu máquina.

Cómo funciona GitHub Actions

- **Workflow y evento.** Un workflow es un archivo YAML que dice *cuándo* (`on:`) y *qué* ejecutar. Aquí hay tres: `ci.yml` (reutilizable), `pr.yml` (pull requests) y `cd.yml` (push a `main`).
- **Job y runner.** Un job es un conjunto de pasos que corre en un runner, una máquina virtual limpia (`runs-on: ubuntu-latest`). Los jobs corren en paralelo salvo que uno diga `needs:` de otro.
- **Step y action.** Un paso ejecuta un comando (`run:`) o una action publicada (`uses: actions/checkout@v4`). Las actions son el equivalente de los plugins de Jenkins.
- **Workflows reutilizables.** `pr.yml` y `cd.yml` no repiten el CI: lo llaman con `uses: ./.github/workflows/ci.yml`. Se define una vez y se arregla una vez.
- **Secrets, variables y ambientes.** Los secretos (`secrets.X`) están cifrados y se enmascaran en los logs; las variables (`vars.X`) no son secretas. Un ambiente (`environment:`) agrupa los de un destino y puede exigir una aprobación.
- **Artefactos y resumen.** Lo que un job guarda con `upload-artifact` se descarga desde el run; lo que escribe en `$GITHUB_STEP_SUMMARY` aparece como tabla en la página del run.
- **Concurrencia.** `concurrency` evita que dos runs del mismo grupo corran a la vez: en un PR, un push nuevo cancela el run anterior; en `main`, un despliegue en curso nunca se cancela.

Anatomía de un workflow

Bloque	Para qué sirve	En descuentos-app
`name`	El nombre que aparece en la pestaña Actions.	`CD (CI y despliegue)`
`on`	Los eventos que lo disparan.	`push` a `main`; `pull_request`; `workflow_call`
`permissions`	Qué puede hacer el token del run. Se deja lo mínimo.	`contents: read`; el preview agrega `pull-requests: write`
`concurrency`	Evita runs simultáneos del mismo grupo.	`group: cd-main`
`jobs`	Los trabajos, cada uno con su `runs-on`.	`ci`, `deploy-api`, `deploy-web`, `smoke-test`
`needs`	Orden: este job espera a aquellos.	`deploy-web` espera a `deploy-api`, que espera a `ci`
`strategy.matrix`	Repite el job con varias versiones.	Java 17 y 21; Node 22 y 24
`steps`, `uses`, `run`	Los pasos del job.	`actions/checkout@v4`, `mvn -B verify`

Bloque	Para qué sirve	En descuentos-app
`env`, `secrets`, `vars`	Variables de entorno, secretos y variables del repositorio.	`VERCEL_TOKEN`, `RENDER_DEPLOY_HOOK`, `API_URL`
`if`	Condición para ejecutar un job o paso.	`if: always()` publica el resumen aunque fallen las pruebas
`environment`	El ambiente de despliegue.	`api-production`, `web-production`, `preview`

Los resultados de un run

Resultado	Color	Qué significa	Causa típica
Success	Verde	Todos los jobs terminaron bien.	Todo en orden… si las pruebas se ejecutaron de verdad.
Failure	Rojo	Un paso devolvió un código distinto de cero.	Una prueba falla, la cobertura no llega, falta un secreto.
Cancelled	Gris	Alguien lo detuvo o `concurrency` lo reemplazó.	Un push nuevo en el mismo PR.
Skipped	Gris claro	El job no corrió porque un `needs` falló o su `if` fue falso.	El E2E cuando falla la API.
Queued / In progress	Amarillo	Espera un runner o está corriendo.	Una etiqueta `runs-on` inexistente lo deja en cola para siempre.

La regla de la práctica: qué es un pipeline sano

Igual que el build sano de la Práctica 6, hay que acordar de antemano qué significa que el pipeline esté bien. Verde no alcanza. Un pipeline sano cumple las diez condiciones siguientes, y contra ellas se evalúan los dos repositorios de la práctica:

Indicador	Condición	Dónde se lee
Resultado	SUCCESS en todos los jobs: `ci`, `deploy-api`, `deploy-web` y `smoke-test`.	Página del run, grafo de jobs.
Pruebas ejecutadas	Las mismas que corren en local (API: 20, front-end: 19) y más de cero.	Resumen del run; log de `mvn` y de Vitest.
Pruebas fallidas	0 fallos y 0 errores.	Resumen del run.
Cobertura	Al menos 80 % de líneas, **exigido por la herramienta** (JaCoCo `check`, `thresholds` de Vitest), no solo medido.	Log del paso; artefactos `api-informes` y `web-informes`.
E2E	Cypress ejecutó sus 5 pruebas y todas pasaron.	Job E2E; artefacto `cypress-evidencia`.
Orden	Nada se despliega antes de que el CI termine bien: `needs` en cada job de despliegue.	Grafo del run.

Indicador	Condición	Dónde se lee
Disparo automático	Un push a `main` o un pull request inicia el run sin un clic.	Cabecera del run: *Triggered via push / pull request*.
Verificación posterior	El smoke test pasa y el commit desplegado es el del run.	Resumen del job `smoke-test`; pie de la aplicación.
Secretos	Ninguno está en el repositorio ni se imprime en los logs.	Búsqueda en el repositorio; log de los pasos.
Hooks	Husky está activo: un mensaje fuera de Conventional Commits se rechaza en local.	`git config core.hooksPath` muestra `.husky/_`.

PARTE A — Práctica guiada: del commit a producción con descuentos-app

Paso 1. El material

Descomprime `practica8-proyectos.zip`. Contiene dos carpetas: `descuentos-app` (el proyecto correcto, que usarás en toda la Parte A) y `reto-cicd` (el mismo proyecto con defectos sembrados, para la Parte B). No abras todavía la segunda.

`descuentos-app` es la *Calculadora de Descuentos* de la Práctica 6, ahora como aplicación web de dos servicios. Las reglas de negocio son las mismas: porcentaje entre 0 y 100, precio mayor que cero y redondeo a dos decimales con la mitad hacia arriba.

Carpeta / archivo	Qué contiene
`api/`	Spring Boot 3 con Java 17. `POST /api/descuento`, `GET /api/info` (versión y commit desplegado) y `/actuator/health`. 20 pruebas JUnit 5, JaCoCo con mínimo de 80 % y un `Dockerfile` que usa Render.
`web/`	Vite, React y TypeScript. 19 pruebas con Vitest, ESLint, Prettier y 5 pruebas E2E con Cypress. `vercel.json` desactiva los despliegues automáticos de Vercel para que solo despliegue el pipeline.
`.husky/`	Los tres hooks: `pre-commit` (lint-staged), `commit-msg` (commitlint) y `pre-push` (pruebas rápidas).
`commitlint.config.mjs`	Reglas de Conventional Commits.
`.github/workflows/ci.yml`	El CI reutilizable: `commits`, `api` (Java 17 y 21), `web` (Node 22 y 24) y `e2e`.
`.github/workflows/pr.yml`	En cada pull request: llama al CI y, si pasa, publica un preview en Vercel y lo comenta en el PR.
`.github/workflows/cd.yml`	En cada push a `main`: llama al CI y, si pasa, despliega la API en Render, el front-end en Vercel y ejecuta el smoke test.
`scripts/`	`verificar-config.sh` (avisa qué secreto falta), `resumen-pruebas.py` (resumen y guardia de cero pruebas) y `smoke.sh` (verificación del despliegue).

Paso 2. Probar en local antes de automatizar

El pipeline va a ejecutar exactamente los mismos comandos que tú. Si no funcionan en tu máquina, no van a funcionar en GitHub. Abre una terminal en `descuentos-app`:

```
cd descuentos-app
mvn -B -f api/pom.xml clean verify
npm --prefix web ci
npm --prefix web run lint
npm --prefix web run test
npm --prefix web run build
```

Al final de `mvn verify` deben aparecer `Tests run: 20, Failures: 0, Errors: 0, Skipped: 0`, el mensaje `All coverage checks have been met.` y `BUILD SUCCESS`. Vitest debe mostrar `Tests 19 passed (19)`. **Anota los números 20 y 19**: son los que el pipeline tiene que mostrar. Captura la salida: es la **Captura 1** del entregable.

Después arranca la aplicación completa. Necesitas dos terminales abiertas a la vez:

```
# terminal 1: la API en http://localhost:8080
java -jar api/target/api.jar

# terminal 2: el front-end en http://localhost:4173
cd web
npm run preview
```

Abre `http://localhost:4173`. Calcula 200 Bs con 25 % (debe dar `Bs 150,00`) y 33,33 Bs con 10 % (`Bs 30,00`). El pie de la página dice `API 1.0.0 · commit desconocido`: en producción ese pie mostrará el commit desplegado. Detén ambos procesos con Ctrl+C.

> **Si `npm ci` tarda:** descarga el navegador de Cypress, unos 200 MB. Es la misma descarga de la Práctica 4; solo ocurre la primera vez.

Paso 3. Husky y commitlint: la primera barrera

Los hooks son scripts que Git ejecuta en momentos fijos. Husky los guarda dentro del repositorio (`.husky/`) para que todo el equipo use los mismos. Primero crea el repositorio local y luego instala las dependencias de la raíz, que activan los hooks:

```
git init -b main
npm install
git config core.hooksPath
```

El último comando debe mostrar `.husky/_`: así Git sabe dónde están los hooks. Los activó el script `prepare` del `package.json` de la raíz, que ejecuta `husky` después de cada `npm install`.

Hook	Cuándo corre	Qué ejecuta aquí	Si falla
`pre-commit`	Antes de crear el commit	`lint-staged`: ESLint y Prettier **solo** sobre los archivos `.ts` y `.tsx` que vas a commitear	No se crea el commit
`commit-msg`	Con el mensaje ya escrito	`commitlint`: el mensaje debe ser `tipo: descripción`	No se crea el commit
`pre-push`	Antes de subir	`npm --prefix web test` y `mvn test` de la API	No se sube nada

Pruébalos. Primero un mensaje que no sigue la convención:

```
git add .
git commit -m "arreglos varios"
```

commitlint lo rechaza con `subject may not be empty` y `type may not be empty`, y no se crea el commit. Captura el rechazo: es la **Captura 2**. Luego crea el commit bueno (los hooks de la raíz y de `web/` se ejecutan sobre los archivos en staging, así que puede tardar unos segundos):

```
git commit -m "feat: proyecto inicial de la práctica 8"
```

Para ver `pre-commit` en acción, crea un archivo con un error de lint, por ejemplo `web/src/malo.ts` con `var y = 2` y una variable sin usar. Al intentar commitearlo, ESLint lo rechaza y el commit no se crea. Borra el archivo antes de seguir (`git restore --staged web/src/malo.ts` y elimínalo).

> **Los hooks se pueden saltar:** `git commit --no-verify` los ignora. Eso no es un fallo de Husky: es la razón por la que el servidor repite las comprobaciones. En la Parte A, Paso 11, verás las dos cosas juntas.

Paso 4. Subir descuentos-app a GitHub

En GitHub crea un repositorio **público** y vacío llamado `descuentos-app` (sin README, sin `.gitignore`, sin licencia: ya vienen en el material). Luego, desde la carpeta del proyecto:

```
git remote add origin https://github.com/TU-USUARIO/descuentos-app.git
git push -u origin main
```

Antes de subir, el hook `pre-push` ejecuta las 19 pruebas del front-end y las 20 de la API: tarda cerca de un minuto. Es el mismo trabajo que hará el servidor, pero en tu máquina y antes de ocupar a nadie. Comprueba en GitHub que `.github/workflows/` está en la raíz y que no se subió ninguna carpeta `node_modules` ni `target`.

Paso 5. El primer run: el CI en verde y el despliegue en rojo

El push a `main` disparó el workflow `CD (CI y despliegue)` sin que hicieras nada. Abre la pestaña **Actions** del repositorio y entra al run. Todavía no configuraste Render ni Vercel, así que el resultado esperado es este:

Job	Resultado esperado	Qué mirar
`ci / Mensajes de commit`	Omitido (*skipped*)	Solo corre en pull requests.
`ci / API (Java 17)` y `(Java 21)`	Verde	En el resumen: 20 pruebas, 0 fallos. La cobertura se exige dentro del paso `mvn verify`.
`ci / Web (Node 22)` y `(Node 24)`	Verde	En el resumen: 19 pruebas, 0 fallos.
`ci / E2E (Cypress)`	Verde	Cypress ejecutó 5 pruebas contra la API y el front-end levantados dentro del runner.
`deploy-api`	**Rojo**	Error: `Falta configuracion: No hay valor para RENDER_DEPLOY_HOOK` y para `API_URL`.
`deploy-web`, `smoke-test`	Omitidos	Esperan a `deploy-api`.

Ese rojo es **deliberado**: el pipeline no puede desplegar porque faltan los secretos, y lo dice con un mensaje claro en lugar de fallar más adelante con un error de la plataforma. Es el trabajo de `scripts/verificar-config.sh`. Al final de la página del run, en **Artifacts**, descarga `api-informes` y abre `jacoco/index.html`: es el informe de cobertura. Captura el run con el CI en verde y el despliegue en rojo: **Captura 3**.

> **Para leer cualquier run:** empieza por el grafo (qué jobs corrieron y en qué orden), luego abre el job en rojo, luego el paso en rojo y busca la **primera** línea de error. Las siguientes suelen ser consecuencia.

Paso 6. Crear el servicio de la API en Render

1. En Render: **New → Web Service → Build and deploy from a Git repository** y elige `descuentos-app`.
2. Name: `descuentos-api`. Branch: `main`. **Root Directory: `api`**. Language/Runtime: **Docker**. Instance Type: **Free**.
3. En **Environment Variables** agrega `CORS_ORIGINS` con el valor `https://descuentos-web*.vercel.app`. Es el patrón de los dominios de Vercel de tu proyecto (producción y previews); lo afinarás en el Paso 7.
4. En **Advanced**: **Health Check Path** `/actuator/health` y **Auto-Deploy: No**. El despliegue lo pedirá el pipeline, no un push; si Render desplegara solo, el smoke test no podría saber qué despliegue esperar.
5. Pulsa **Create Web Service**. Render construye la imagen con el `Dockerfile` de `api/` (Maven descarga dependencias: la primera vez tarda entre 5 y 10 minutos). Cuando termine, `https://descuentos-api-XXXX.onrender.com/api/info` debe responder un JSON con `app`, `version` y `commit`.
6. En el servicio: **Settings → Deploy Hook**. Copia la URL completa (lleva un `key=` que es un secreto) y la URL pública del servicio.

> **Si Render no encuentra el Dockerfile:** la ruta se interpreta relativa al Root Directory. Deja `./Dockerfile`; si el error persiste, prueba `api/Dockerfile`.

Paso 7. Crear el proyecto en Vercel

1. En Vercel: **Add New → Project**, importa `descuentos-app`. **Root Directory: `web`**. Framework Preset: Vite (se detecta solo). Nombre del proyecto: `descuentos-web` (agrega tu usuario si ya existe). Pulsa **Deploy**.
2. En el proyecto: **Settings → Deployment Protection → Vercel Authentication: Disabled**. Si queda activa, el smoke test del preview recibe una página de login (401) en lugar de la aplicación.
3. Crea un token: tu avatar → **Account Settings → Tokens → Create**. Alcance: tu cuenta; vencimiento: 30 días. Cópialo: solo se muestra una vez.
4. Obtén los dos identificadores. En la carpeta `descuentos-app` ejecuta `npx vercel link` e inicia sesión; elige tu cuenta y **enlaza con el proyecto existente** `descuentos-web`. Se crea `.vercel/project.json` con `orgId` y `projectId` (esa carpeta está en `.gitignore`: **no la subas**).
5. Anota el dominio de producción del proyecto (Settings → Domains), por ejemplo `https://descuentos-web.vercel.app`.
6. Vuelve a Render y ajusta `CORS_ORIGINS` al nombre real del proyecto si es distinto (`https://TU-PROYECTO*.vercel.app`). Render reinicia el servicio al guardar.

> **Qué hace el patrón de CORS:** el navegador solo deja que una página de un dominio llame a la API de otro si la API lo autoriza. El * cubre el dominio de producción y los previews (`descuentos-web-git-rama-usuario.vercel.app`) sin abrir la API a cualquier sitio, como pasaría con `https://*.vercel.app`.

Paso 8. Secretos, variables y ambientes en GitHub

En el repositorio: **Settings → Secrets and variables → Actions**. Los nombres deben coincidir **exactamente** con los de los workflows (distinguen mayúsculas).

Nombre	Pestaña	Valor
`VERCEL_TOKEN`	Secrets	El token del Paso 7.
`VERCEL_ORG_ID`	Secrets	`orgId` de `.vercel/project.json`.
`VERCEL_PROJECT_ID`	Secrets	`projectId` de `.vercel/project.json`.
`RENDER_DEPLOY_HOOK`	Secrets	La URL del Deploy Hook del Paso 6.
`API_URL`	Variables	URL pública de la API, sin `/` final: `https://descuentos-api-XXXX.onrender.com`.
`WEB_URL`	Variables	Dominio de producción del front-end, sin `/` final.

Luego, en **Settings → Environments**, crea tres ambientes: `preview`, `api-production` y `web-production`. No hace falta configurarlos todavía; existen para que GitHub registre cada despliegue en la pestaña *Deployments* y, en el Ejercicio 3, para exigir una aprobación.

> **Un secreto es un secreto:** nunca lo pegues en un workflow, en el código, en un README ni en una captura. GitHub enmascara su valor en los logs, pero solo si lo ve tal cual: un secreto transformado (por ejemplo, codificado en base64) no se enmascara. Si se te escapa uno, revócalo y crea otro; borrar el commit no alcanza.

Paso 9. El despliegue completo

En la pestaña Actions abre el run del Paso 5 y pulsa **Re-run failed jobs**. Ahora el pipeline hace, en este orden:

1. `deploy-api` verifica los secretos, llama al Deploy Hook de Render con `&ref=` y el SHA del commit y espera, consultando `/api/info` cada 10 segundos, a que la API informe **ese** commit. Si Render tarda más de 15 minutos, el job falla.
2. `deploy-web` ejecuta `vercel deploy --prod` con `VITE_API_URL` y `VITE_COMMIT` como variables de build: el front-end sale apuntando a tu API y con el commit grabado en su HTML.
3. `smoke-test` ejecuta `scripts/smoke.sh` y escribe una tabla en el resumen del run.

Comprobación del smoke test	Qué detecta
La API responde `/actuator/health` = UP	Servicio caído o aún dormido (reintenta hasta 2 minutos).
La API corre el commit del run	Render no desplegó lo que se pidió: sigue corriendo la versión anterior.
`POST /api/descuento`: 200 Bs con 25 % = 150	La API arranca pero calcula mal.
CORS: la API acepta el origen del front-end	El front-end desplegado no podría llamar a la API desde el navegador.
El front-end responde y es el commit del run	Vercel sirve una versión vieja o un error.

Cuando los cuatro jobs estén en verde, abre `WEB_URL`. El pie de la página debe decir `API 1.0.0 · commit` seguido de los primeros 7 caracteres del SHA del último commit. Calcula un descuento. Captura el grafo del run en verde con el resumen del smoke test: **Captura 4**; y la aplicación en producción con el pie visible: **Captura 5**.

> **La primera vez es lenta:** Render construye la imagen desde cero y el servicio gratuito despierta tras minutos de inactividad. Los siguientes despliegues son más rápidos. Si el smoke test falla en CORS, revisa `CORS_ORIGINS` (Paso 7, punto 6).

Paso 10. Leer un run

Pantalla	Dónde	Qué debes ver en descuentos-app
Grafo de jobs	Página del run	`ci` (con sus jobs anidados) → `deploy-api` → `deploy-web` → `smoke-test`, con flechas.
Resumen	Página del run, sección Summary	Las tablas de pruebas (20 y 19) y la del smoke test.
Log de un job	Clic en el job	Cada paso se expande; `mvn -B verify` muestra `Tests run: 20` y `All coverage checks have been met.`
Artefactos	Final de la página del run	`api-jar`, `api-informes`, `web-informes`, `cypress-evidencia`.
Deployments	Página principal del repositorio	Los despliegues a `api-production` y `web-production`, con su URL.
Evento	Cabecera del run	*Triggered via push*, con el commit y el autor.

Compara los números del resumen con los que anotaste en el Paso 2. Tienen que coincidir. Esta comprobación, que parece trivial, es la que en la Parte B separa un pipeline sano de uno que miente.

Paso 11. Romper el código y ver el pipeline en rojo

Un pipeline que nunca se pone rojo no demuestra nada. Vas a introducir un defecto a propósito, pero esta vez por la vía correcta: en una rama y con un pull request.

```
git switch -c fix/limite-descuento
```

En `api/src/main/java/bo/edu/usfx/descuentos/CalculadoraDescuentos.java` cambia la validación del porcentaje, de modo que rechace el 100 %:

```
// ANTES
if (porcentajeDescuento < 0 || porcentajeDescuento > 100) {
// DESPUES (defecto introducido a proposito)
if (porcentajeDescuento < 0 || porcentajeDescuento >= 100) {

git commit -am "fix: ajusta el limite del descuento"
git push -u origin fix/limite-descuento
```

El mensaje es válido y `pre-commit` no tiene nada que objetar, así que el commit se crea. Pero el **`pre-push` ejecuta las pruebas y falla**: `descuentoTotalDejaElPrecioEnCero` no pasa y el push se cancela. El hook acaba de proteger al servidor de tu error. Para ver la segunda barrera, salta el hook a propósito:

```
git push --no-verify -u origin fix/limite-descuento
```

En GitHub abre un **pull request** hacia `main`. Se dispara PR `(CI y preview)`: el job API termina en rojo con la prueba que falla, E2E se omite y **`preview` ni siquiera arranca**, porque depende de `ci`. El PR muestra la marca roja y nadie desplegó nada. Captura el PR con el CI en rojo: **Captura 6**.

Ahora corrige. Revierte el cambio (deja la línea como estaba), haz commit con un mensaje válido y sube sin saltar nada:

```
git commit -am "fix: corrige el limite del descuento"
git push
```

El push nuevo **cancela** el run anterior del PR (`concurrency`) y arranca otro. Esta vez el CI termina en verde, `preview` publica la aplicación en una URL propia y el bot deja un comentario en el PR con el enlace. Ábrela, prueba el 100 % de descuento y captura el PR con el comentario del preview: **Captura 7**.

Por último, haz **merge** del pull request. El push resultante a `main` dispara `CD (CI y despliegue)`: CI, despliegue de la API, despliegue del front-end y smoke test. Cuando termine, el pie de la aplicación en producción muestra el commit del merge. Captura el run: **Captura 8**.

> **Un detalle del merge:** con *Squash and merge* el mensaje del commit en `main` será el título del PR, así que debe seguir la convención (`fix: ...`). El job `Mensajes de commit` valida los commits del PR, no el del merge.

Paso 12. Cómo se interpreta un run fallido

Un run rojo no se arregla mirando el icono. Se lee el log de arriba hacia abajo buscando la primera línea de error y se ubica el problema en una de cuatro capas, de afuera hacia adentro:

1. **Hooks locales.** Husky no está instalado o rechaza lo que no debe. Se ve en tu terminal, antes de que exista un run.
2. **El workflow.** Una versión equivocada, un `needs` que falta, un nombre de secreto mal escrito, un comando que no existe en el runner. El run falla en segundos, a veces sin ejecutar ningún paso de tu código. Se corrige con un commit al YAML.
3. **El proyecto y el código.** El `pom.xml`, el `package.json`, un puerto, una prueba que falla. El mismo problema aparece en local con el mismo comando.
4. **La plataforma.** Render o Vercel: un token vencido, un Root Directory mal escrito, `CORS_ORIGINS` sin el dominio, un servicio dormido. Se corrige en el panel de la plataforma, no en el repositorio.

La pregunta que decide entre las capas 1 a 3 y la 4 es siempre la misma: **¿se reproduce en local con el mismo comando?** Si `mvn -B verify` falla igual en tu máquina, el problema es del proyecto o del código. Si en tu máquina funciona y en GitHub no, es del workflow o de la plataforma.

PARTE B — Reto guiado: el pipeline que miente

Hasta aquí construiste un pipeline que funciona. En esta parte recibes uno que alguien más escribió: `reto-cicd`, la misma aplicación con sus workflows. El equipo anterior asegura que "el pipeline está en verde". La carpeta tiene **ocho defectos sembrados**: tres en la forma en que el workflow ejecuta y trata las pruebas, uno en el orden y otro en los secretos del despliegue, dos en el código y uno en la configuración del E2E. Algunos se ven con solo mirar el color del run; otros producen runs **verdes** que no prueban nada. Tu trabajo es encontrarlos todos, uno por uno, sin tocar las pruebas.

Las reglas de negocio son las de la Parte A. En este repositorio el despliegue es **simulado** (`scripts/deploy-simulado.sh`): no toca Render ni Vercel, solo comprueba que el workflow recibe sus secretos y respeta el orden.

Paso 1. Subir el reto y preparar los secretos

1. Crea en GitHub un repositorio público `descuentos-reto` y sube el contenido de `reto-cicd` igual que en el Paso 4 de la Parte A (rama `main`). Los hooks de Husky te acompañan: instala con `npm install` en la raíz antes del primer commit.
2. Crea cuatro secretos del repositorio con **cualquier valor** (por ejemplo `simulado`): `RENDER_DEPLOY_HOOK`, `VERCEL_TOKEN`, `VERCEL_ORG_ID` y `VERCEL_PROJECT_ID`. Crea también los ambientes `api-production` y `web-production`.

3. Trabaja directamente sobre `main`, como en la Práctica 6: cada push dispara el workflow `CD (CI y despliegue simulado)`.

> **No toques las pruebas ni la plataforma para que el reto pase:** no borres ni comentes pruebas, no cambies los valores esperados, no bajes el umbral de cobertura, no uses `--no-verify` para esquivar un hook y no renombres los secretos para que coincidan con un error. Todos los defectos se corrigen con commits al repositorio. Cada corrección es un commit separado con un mensaje Conventional Commits que diga qué defecto corrige (`fix(ci): defecto 1 ...`): el historial de commits es la evidencia de tu diagnóstico.

Paso 2. Los síntomas

Los defectos no aparecen todos a la vez: cada uno esconde al siguiente. Esta es la secuencia que vas a observar si corriges **uno por commit y en orden** (valores del entorno del docente):

Run	Síntoma	Dónde se ve
1	El job `API` falla en rojo (Java 17 y 21) en pocos segundos. `Web` queda **verde**. E2E omitido. **`deploy-api` corre en verde aunque el CI falló.** `deploy-web` falla.	Log de `API`: error de versión de Java (`class file version 61.0` o `release version 17 not supported`). Grafo: `deploy-api` sin flecha desde `ci`. `deploy-web`: `No hay valor para VERCEL_TOKEN`.
2	`API` en **verde**, pero sin ninguna línea `Tests run:`. `Web` sigue en verde. E2E ahora corre y falla.	Log de `mvn`: `Tests are skipped.` y `Skipping JaCoCo execution due to missing execution data file`. E2E: `cy.visit() failed trying to load: http://localhost:5173`.
3	`API` en rojo con 2 pruebas fallidas. `Web` sigue en **verde**. E2E omitido.	Log de `mvn`: `Tests run: 20, Failures: 2`. Log del paso de pruebas de `Web`: `Tests 3 failed | 16 passed`, y el job en verde.
4	`API` y `Web` en rojo: la API con 2 pruebas fallidas y el front-end con 3.	Log de Vitest: `rechaza el porcentaje 100.01`, `rechaza el porcentaje 150` y `no llama a la API si el dato es invalido`.
5	`API` en verde con 20 pruebas. `Web` en rojo con 3 pruebas fallidas.	Resumen de la API: 20 pruebas, 0 fallos. Log de Vitest.
6	`API` y `Web` en verde. E2E corre y falla.	E2E: `cy.visit() failed trying to load: http://localhost:5173/` y `ECONNREFUSED`.
7	`ci` completo en verde (`Tests run: 20`, `19 passed`, 5 pruebas de Cypress). `deploy-api` sigue corriendo en paralelo al CI. `deploy-web` en rojo.	Grafo del run. `deploy-web`: `No hay valor para VERCEL_TOKEN`.
8	`deploy-api` ahora espera al CI. `deploy-web` sigue en rojo.	`deploy-web`: `No hay valor para VERCEL_TOKEN`, aunque el secreto existe.
9	Todo en verde: CI completo, `deploy-api`, `deploy-web`.	Grafo del run; resumen con 20 y 19 pruebas.

Fíjate en el job `API` del run 2 y en el job `Web` de los runs 1 a 3: son verdes. Si el equipo anterior miraba solo el color, tenía razón: “el pipeline está en verde”. Contra la regla del pipeline sano de la Sección 3 incumplen indicadores cada uno. Antes de tocar nada, completa esta tabla en tu informe, una fila por defecto:

Columna	Qué escribir
Síntoma observado	Qué run, qué job y qué línea del log, tal como aparece.
¿Se reproduce en local?	Sí o no, con el comando que usaste. Decide la capa.
Capa	Hooks locales, workflow, proyecto o código, plataforma (Parte A, Paso 12).
Indicador violado	Cuál de las diez filas de la Sección 3 o de las reglas de negocio se incumple.
Archivo y línea	Dónde está la causa.
Corrección aplicada	La línea antes y después, el hash del commit y el número del run que lo confirma.

Paso 3. Método de trabajo: cuatro preguntas

Para cada síntoma, responde en este orden y no saltes ninguna:

1. **¿Qué job falla y cuál es la primera línea de error?** No digas "el pipeline falla": di "el run 1 falla en el job `API (Java 17)`, en el paso `Compilar y verificar`, con `UnsupportedClassVersionError`".
2. **¿Se reproduce en local con el mismo comando?** Ejecuta en tu máquina lo que ejecuta el paso. Si en local funciona, el problema está en el workflow o en la plataforma; si falla igual, en el proyecto o en el código.
3. **¿En qué capa está?** La respuesta dice qué archivo abrir.
4. **¿Cuál es la corrección mínima?** Cambia lo justo, haz un commit, deja que el push dispare el run y confirma que el síntoma desapareció y que no apareció otro que no esperabas.

> **Un defecto, un commit, un run:** los ocho defectos se encadenan: mientras la API no compile no sabrás si las pruebas corren; mientras las pruebas se omitan, no sabrás cuáles fallan; mientras el front-end ignore sus fallos, el job estará verde. Si corriges tres cosas en un commit y el run sale verde, no puedes demostrar qué arregló qué, y la rúbrica pide un commit por defecto.

Paso 4. Defecto 1 resuelto — la versión de Java

Este va resuelto como ejemplo del método. El run 1 termina en rojo en el job `API` y la consola dice, en esencia:

```
Run mvn -B -DskipTests verify
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-
plugin:3.5.6:build-info (default) on project descuentos-api:
        Execution default of goal ...:build-info failed: ...
UnsupportedClassVersionError ...
        has been compiled by a more recent version of the Java Runtime (class
file version 61.0),
        this version of the Java Runtime only recognizes class file versions up
to 55.0
Error: Process completed with exit code 1.
```

Pregunta 1 — job y primera línea. Falla `API`, paso `Compilar y verificar`. La línea que importa es `class file version 61.0 … up to 55.0`: Java 61 es la versión 17, Java 55 es la 11. Según qué plugin falle primero el mensaje puede decir en cambio `release version 17 not supported`; es el mismo problema.

Pregunta 2 — ¿se reproduce en local? No: en tu máquina tienes Java 17 o superior y `mvn -B verify` funciona. El problema está en el entorno donde corre el workflow.

Pregunta 3 — ¿qué capa? El workflow. Abre `.github/workflows/ci.yml` y busca `setup-java`: el job pide `java-version: '11'` fijo, mientras el `pom.xml` exige 17. Compara con el `ci.yml` de la Parte A: allí es `${{ matrix.java }}`. Fíjate también en que fallan los dos jobs de la matriz (Java 17 y 21): la matriz existe, pero el valor fijo la anula.

```
# ANTES (defectuoso)
- uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: '11'

# DESPUES (corregido)
    java-version: ${{ matrix.java }}
```

Pregunta 4 — corrección mínima. Una línea del workflow. Commit con el mensaje `fix(ci): defecto 1 - java-version usa la matriz` y push. El run 2 arranca solo. Fíjate en lo que pasó con la **configuración de GitHub**: no hizo falta tocar nada en Settings.

> **Lo que acaba de pasar:** este defecto no lo detecta ninguna prueba de las Prácticas 1 a 6, porque no está en el código: está en el contrato entre el workflow y el runner. Por eso se lee el log y no solo el icono, y por eso la primera pregunta siempre es "¿en local funciona?".

Paso 5. Los siete defectos restantes

Cada síntoma de la tabla del Paso 2 corresponde a un defecto distinto. Estas pistas te dicen dónde mirar, no qué escribir.

Síntoma	Pista
Defecto 2 · Run 2: `API` verde, sin `Tests run:`, `Tests are skipped.`	Lee el comando del paso completo, hasta el final. Hay una opción de Maven que omite las pruebas. Mira también por qué JaCoCo no se queja: sin pruebas no hay datos de cobertura que comprobar. Compara con el `ci.yml` de la Parte A.
Defecto 3 · Runs 1 a 3: `Web` verde con `3 failed` en el log	Un paso con un atributo que le dice a GitHub "si falla, sigue como si nada". Busca en el paso de pruebas del job `web`. Al quitarlo, el job pasará a rojo y aparecerán los defectos del código del front-end.
Defecto 4 · Run 3: `Failures: 2` en la API	Regla de negocio "precio inválido". Abre `CalculadoraDescuentos.java` y compara la validación del precio con la regla: ¿menor que cero o menor o igual que cero? No toques las pruebas.
Defecto 5 · Run 4: tres pruebas del front-end en rojo	Regla "porcentaje inválido". Mira `validar` en `web/src/descuentos.ts` y compárala con la validación de la API: ¿qué valor se deja pasar?
Defecto 6 · Runs 2 y 6: `cy.visit()` falla con `ECONNREFUSED` en el puerto 5173	¿En qué puerto sirve `npm run preview` (mira `package.json`)? ¿Qué puerto espera `wait-on` en el workflow? ¿Qué puerto dice `baseUrl` en `cypress.config.ts`? Dos de los tres coinciden.
Defecto 7 · Runs 1 a 7: `deploy-api` corre en paralelo al CI	Mira el grafo del run y compáralo con el de la Parte A: ¿qué flecha falta? ¿Qué palabra clave del workflow ordena los jobs? Es la corrección más peligrosa de ignorar: una API rota llegaría a producción.

Síntoma	Pista
Defecto 8 · Runs 1 a 8: `No hay valor para VERCEL_TOKEN`	El secreto existe en Settings. El mensaje sale de `scripts/verificar-config.sh`. Compara carácter por carácter el nombre que lee el workflow con el que creaste. Se corrige **en el workflow**, no en Settings.

Verificación de la Parte B: cuando el repositorio `descuentos-reto` cumpla las diez filas del pipeline sano que le aplican (Success en todos los jobs, 20 y 19 pruebas ejecutadas, 0 fallos, cobertura exigida, E2E con 5 pruebas, `deploy-*` solo después de `ci`, disparo automático y secretos presentes), captura el grafo del run con los resúmenes: es la **Captura 9**. Y antes de entregar, haz un último commit trivial (por ejemplo, una línea en el README) y comprueba que el run arranca solo y termina en verde.

PARTE C — Ejercicios propuestos

Ejercicio 1 — Terminar el reto y poner un guardia contra las pruebas ausentes

Corrige los siete defectos restantes de `reto-cicd` hasta que el pipeline cumpla el estándar sano, y documenta cada uno en la tabla de seis columnas del Paso 2 de la Parte B, con el fragmento antes y después, el hash del commit y el número del run que confirma la corrección.

Después, haz que el defecto 2 **no pueda volver a pasar desapercibido**. Copia `scripts/resumen-pruebas.py` de `descuentos-app` al reto y agrega al job `api` y al job `web` los pasos que lo ejecutan (los encuentras en `ci.yml` de la Parte A). Demuestra que funciona: vuelve a poner `-DskipTests` en un commit de prueba, comprueba que el run se pone **rojo** con `Cero pruebas ejecutadas` y revierte el commit.

Ejercicio 2 — Proteger la rama main

Hasta ahora podías hacer push directo a `main` con un workflow en rojo. En `descuentos-app`, ve a **Settings → Branches → Add branch ruleset** (o *branch protection rule*) sobre `main` y exige: pull request antes de fusionar, que los *status checks* pasen y que la rama esté al día. Los checks requeridos aparecen con el prefijo del job que llama al CI: `ci / API (Java 17)`, `ci / Web (Node 22)`, `ci / E2E (Cypress)` y `ci / Mensajes de commit`.

Demuestra que funciona con un pull request que rompa una prueba: el botón de merge debe quedar bloqueado. Entrega las capturas del PR bloqueado y del PR desbloqueado, y responde en un párrafo: ¿qué cubre esta regla que no cubre Husky?

Ejercicio 3 — Aprobación manual para producción

En **Settings → Environments → web-production**, agrega un **Required reviewer** (tú mismo). Haz un merge a `main` y observa que el run se detiene en `deploy-web` con el mensaje *Waiting for review*: la API ya se desplegó, el front-end espera. Aprueba el despliegue desde la página del run. Con esto el pipeline pasa de despliegue continuo a entrega continua.

Entrega la captura del run en espera y responde: ¿qué riesgo mitiga la aprobación y cuál introduce? ¿Qué pasa con la API ya desplegada si rechazas el despliegue del front-end? Propón cómo reordenarías los jobs para que no quede una API nueva con un front-end viejo incompatible.

Ejercicio 4 — Rollback

Despliega un cambio que **pase todas las pruebas pero sea incorrecto de negocio** (por ejemplo, cambia el redondeo a `RoundingMode.DOWN` y ajusta la prueba que lo comprueba: es un error que el pipeline no puede ver). Una vez en producción, vuelve a la versión anterior de tres maneras y mide cada una con cronómetro, desde que decides volver hasta que el pie de la aplicación muestra el commit anterior:

- **Revertir el commit:** `git revert` en una rama, pull request y merge; el pipeline completo despliega la versión anterior.
- **Redesplegar un commit anterior desde Actions:** ejecuta CD con *Run workflow* sobre un commit anterior si adaptas el workflow para aceptar un `ref`.
- **Rollback en las plataformas:** *Promote to Production* de un despliegue anterior en Vercel y *Rollback* en Render.

Entrega una tabla con el tiempo, los pasos y el riesgo de cada una, y responde: ¿cuál usarías en un incidente real y por qué? ¿Qué pasa con una migración de base de datos en cada una?

Ejercicio 5 — Integrar una práctica anterior

Agrega al pipeline de `descuentos-app` una etapa que automatice una de las prácticas anteriores. Elige una:

- **Análisis estático (Práctica 3).** Crea un proyecto en SonarCloud (gratuito para repositorios públicos), guarda el token como secreto `SONAR_TOKEN` y agrega un job `analisis` que ejecute `mvn -B verify sonar:sonar` en la API con `-Dsonar.organization` y `-Dsonar.projectKey`. Activa el *Quality Gate* como requisito y demuestra un run en rojo provocado con un *code smell* grave.
- **Rendimiento (Práctica 5).** Agrega al final de `cd.yml` un job `rendimiento` que descargue JMeter, ejecute un plan de humo contra `API_URL` en modo no-GUI y falle el run si el percentil 95 supera el SLA que definiste en la Práctica 5. Considera la demora de un servicio gratuito dormido: ¿cómo evitas que la primera petición lenta falle la prueba?

Entrega los workflows finales, una captura del run con la nueva etapa en verde y otra con la etapa en rojo provocada a propósito.

Ejercicio 6 — Análisis escrito

Media página por pregunta, con ejemplos concretos de tu propio trabajo en esta práctica:

1. El job `API` del run 2 y el job `Web` de los runs 1 a 3 del reto fueron verdes. ¿Qué habría pasado en un equipo que solo mira el color del run? ¿Qué indicador del pipeline sano los habría detectado primero? Relaciónalo con los builds 3 y 4 del reto de la Práctica 6.
2. De los ocho defectos del reto, ¿cuáles habría detectado un desarrollador con los hooks de Husky y cuáles solo aparecen en el servidor? ¿Por qué el CI repite la validación de commitlint?
3. Tu `VERCEL_TOKEN` queda guardado como secreto. Describe tres maneras en que podría filtrarse (logs, un pull request, un colaborador, un paso mal escrito) y cómo protege cada defensa que usó este pipeline (enmascarado, `permissions` mínimos, ambientes, tokens con vencimiento). ¿Qué pasa con los secretos en un pull request que viene de un fork?
4. ¿Qué política propondrías sobre cuándo se permite saltar un control (`--no-verify`, `continue-on-error`, bajar el umbral de cobertura, aprobar un despliegue sin mirar)? Relaciónala con lo discutido en la Práctica 3 sobre apagar reglas, en la Práctica 4 sobre modificar specs, en la Práctica 5 sobre subir umbrales y en el Plan de Calidad de la Práctica 7.

4. Entregable

Sube a Classroom, dentro de la carpeta `practica8` de tu repositorio:

- **PDF con las Capturas 1 a 9** (pruebas en local, hook rechazando un commit, primer run con el despliegue en rojo, run completo en verde con el smoke test, aplicación en producción con el pie visible, PR en rojo, PR en verde con el preview, run del merge a `main`, reto sano).
- En el mismo PDF: la **tabla de seis columnas** con los ocho defectos del reto y su corrección, con código antes y después, hash del commit y número de run.
- En el mismo PDF: las capturas y respuestas de los Ejercicios 2 a 5 y el análisis escrito del Ejercicio 6.
- Los **enlaces** a tus dos repositorios públicos (`descuentos-app` y `descuentos-reto`) y a la aplicación desplegada. El historial de commits del reto debe mostrar un commit por defecto.

> **Verificación final:** antes de entregar, haz un último commit trivial en cada repositorio y comprueba que los dos runs arrancan solos y terminan en verde con el número de pruebas correcto. Si para que el reto pasara modificaste o borraste pruebas, bajaste el umbral de cobertura o usaste `--no-verify`, el reto no cuenta. **Revoca el token de Vercel** al terminar la práctica si no vas a seguir usándolo.

5. Rúbrica de evaluación

Criterio	Puntos
Husky y commitlint activos: `core.hooksPath` correcto y un commit rechazado y otro aceptado (Capturas 1 y 2)	10
Repositorio en GitHub y CI en verde con 20 y 19 pruebas, cobertura exigida y artefactos (Capturas 1 y 3)	10
Render y Vercel configurados; secretos, variables y ambientes; despliegue completo con smoke test (Capturas 4 y 5)	15
Ciclo con pull request: rojo → verde con preview y despliegue a producción al fusionar (Capturas 6, 7 y 8)	10
Calidad del pipeline: CI reutilizable, `needs`, `permissions` mínimos, sin secretos en el repositorio, commits convencionales	5
Los 8 defectos de `reto-cicd` corregidos: pipeline sano sin modificar pruebas ni umbrales (Captura 9)	20
Tabla de diagnóstico de los 8 defectos, completa, con capa, commit y número de run	10
Ejercicio 1: guardia de cero pruebas demostrado	3
Ejercicio 2: protección de `main` con PR bloqueado y desbloqueado	3
Ejercicio 3: aprobación manual y respuesta sobre el orden de los jobs	3
Ejercicio 4: tabla de rollback con tiempos y análisis	3
Ejercicio 5: etapa de análisis estático o de rendimiento integrada, en verde y en rojo	3
Ejercicio 6: análisis escrito y presentación del informe	5
TOTAL	**100**

> **Cómo se pierden puntos rápido:** entregar un pipeline verde que no ejecuta pruebas, modificar o borrar pruebas del reto, usar `--no-verify` para esquivar un hook, corregir los ocho defectos en un solo commit, pegar tokens o URLs de deploy hooks en un workflow, subir `node_modules`, `target` o la carpeta `.vercel`, o dejar un secreto visible en una captura.

6. Solución de problemas frecuentes

Síntoma	Causa probable y solución
`git config core.hooksPath` no muestra nada	Husky no se activó: ejecutaste `npm install` antes de `git init`, o no lo ejecutaste en la raíz. Haz `git init -b main` y luego `npm install` (o `npx husky`).
`pre-commit` falla con `eslint: not found`	Faltan las dependencias del front-end: `npm --prefix web ci`.
`pre-push` falla con `mvn: command not found`	Maven no está en el PATH. Verifica con `mvn -v` en la misma terminal.
`commitlint` rechaza `subject may not be empty` / `type may not be empty`	El mensaje no sigue `tipo: descripción`. Ejemplos válidos: `feat: agrega descuento`, `fix(api): corrige el redondeo`. Anexo B.
El workflow no aparece en la pestaña Actions	El archivo no está en `.github/workflows/` de la rama principal, la extensión no es `.yml` o el YAML tiene un error de sintaxis: GitHub lo ignora y lo avisa arriba de la lista.
El job queda en *Waiting for a runner to pick up this job* para siempre	La etiqueta de `runs-on` no existe (por ejemplo `ubuntu-lastest`). Cancela el run y corrige el nombre.
`Unable to resolve action` o `Can't find action`	Nombre o versión inexistente de una action (`actions/checkout@v44`). Usa una versión que exista.
`npm ci` falla con `can only install with an existing package-lock.json`	Falta `package-lock.json` en el repositorio o no coincide con `package.json`. Ejecuta `npm install` en local y haz commit del lock.
`No hay valor para NOMBRE` en `verificar-config.sh`	El secreto o la variable no existe, está en la pestaña equivocada (Secrets contra Variables) o el nombre difiere en una letra del que lee el workflow.
`Resource not accessible by integration` al comentar en el PR	Faltan `permissions: pull-requests: write`, o el PR viene de un fork (no recibe secretos ni permisos de escritura).
Vercel pide enlazar el proyecto o no encuentra el proyecto	Faltan `VERCEL_ORG_ID` o `VERCEL_PROJECT_ID`, o vienen de otro proyecto. Repite `npx vercel link` y copia los valores de `.vercel/project.json`.
El smoke test del preview recibe 401 o una página de login	*Vercel Authentication* está activa. Desactívala en Settings → Deployment Protection.
Render falla con `failed to read dockerfile`	Root Directory mal escrito o Dockerfile Path incorrecto. Root Directory: `api`; Dockerfile Path: `./Dockerfile` o `api/Dockerfile`.

Síntoma	Causa probable y solución
La API tarda en responder o da 502 al principio	Servicio gratuito dormido: tarda cerca de un minuto en despertar. El smoke test reintenta durante 2 minutos; si no alcanza, vuelve a ejecutar el job.
`deploy-api` agota los 15 minutos esperando el commit	Render no desplegó el commit: revisa la pestaña *Events* del servicio, que el Deploy Hook sea el de este servicio y que el commit exista en `main`.
El navegador muestra `blocked by CORS policy`	`CORS_ORIGINS` no incluye el dominio de Vercel o tiene un error de patrón. Corrígelo en Render (se reinicia solo) y recarga.
`Rule violated for bundle … lines covered ratio is 0.xx, but expected minimum is 0.80`	La cobertura bajó del umbral: agregaste código sin pruebas. Escribe las pruebas; no bajes el umbral.
`cy.visit() failed … ECONNREFUSED`	El servidor no está levantado o el puerto del `baseUrl` no es el de `npm run preview` (4173).
`Port 8080 / 4173 is already in use` en local	Queda un proceso de una ejecución anterior. Ciérralo o cambia de terminal.

Anexo A — Comandos usados en la práctica

Comando	Qué hace
`git init -b main`	Crea el repositorio local con la rama `main`.
`npm install` (en la raíz)	Instala Husky, lint-staged y commitlint y activa los hooks (script `prepare`).
`git config core.hooksPath`	Debe mostrar `.husky/_`: confirma que los hooks están activos.
`git commit -m "feat: ..."`	Crea un commit; `pre-commit` y `commit-msg` se ejecutan antes.
`git commit --no-verify` / `git push --no-verify`	Salta los hooks. Úsalo solo para demostrar que el servidor los repite.
`mvn -B -f api/pom.xml clean verify`	Compila, prueba y exige la cobertura mínima, igual que el job `API`.
`npm --prefix web ci`	Instala exactamente lo del `package-lock.json`, como en Actions.
`npm --prefix web run test`	Pruebas del front-end con Vitest.
`npm --prefix web run lint`	Análisis estático con ESLint.
`npm --prefix web run build`	Verifica los tipos con `tsc` y genera `dist/`.
`npm --prefix web run preview`	Sirve el build en `http://localhost:4173`.
`npm --prefix web run e2e`	Ejecuta Cypress; la API y el front-end deben estar corriendo.
`npx vercel link`	Enlaza la carpeta con un proyecto de Vercel y crea `.vercel/project.json` con `orgId` y `projectId`.

Comando	Qué hace
`git switch -c nombre` / `git push -u origin nombre`	Crea una rama y la sube, base de un pull request.
`curl https://TU-API.onrender.com/api/info`	Muestra qué versión y qué commit corre la API.

Anexo B — Conventional Commits y hooks

Un mensaje válido tiene la forma `tipo(alcance opcional): descripción`. El tipo dice qué clase de cambio es y permite, más adelante, generar versiones y registros de cambios de forma automática.

Tipo	Cuándo usarlo	Ejemplo
`feat`	Una funcionalidad nueva.	`feat(api): agrega descuento por cantidad`
`fix`	La corrección de un defecto.	`fix(web): rechaza porcentajes mayores que 100`
`test`	Agregar o ajustar pruebas.	`test(api): cubre el redondeo de 0,5 centavos`
`ci`	Cambios en workflows y scripts del pipeline.	`ci: espera al CI antes de desplegar la API`
`docs`	Solo documentación.	`docs: explica los secretos en el README`
`refactor`	Reordenar código sin cambiar su comportamiento.	`refactor(api): extrae la validación del precio`
`chore` / `build`	Mantenimiento, dependencias, configuración.	`chore: actualiza dependencias de web`

Mensaje	¿Válido?	Por qué
`arreglos varios`	No	No tiene tipo ni descripción separados por `:`.
`Fix: corrige el redondeo`	No	El tipo va en minúsculas.
`fix: corrige el redondeo.`	No	El asunto no puede terminar en punto.
`fix(api): corrige el redondeo`	Sí	Tipo, alcance y descripción.

Hook	Archivo	Contenido
`pre-commit`	`.husky/pre-commit`	`npx lint-staged` (la configuración está en `web/.lintstagedrc.json`)
`commit-msg`	`.husky/commit-msg`	`npx --no -- commitlint --edit "$1"`
`pre-push`	`.husky/pre-push`	Pruebas del front-end y de la API

Anexo C — Plataformas, planes gratuitos y seguridad

Qué necesita cada plataforma

Plataforma	Qué se configura	Qué se copia al repositorio
Render	Web Service Docker, Root Directory `api`, Free, Health Check `/actuator/health`, Auto-Deploy desactivado, variable `CORS_ORIGINS`.	Deploy Hook (secreto `RENDER_DEPLOY_HOOK`) y URL pública (variable `API_URL`).
Vercel	Proyecto con Root Directory `web`, Vercel Authentication desactivada, token de la cuenta.	`VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID` y el dominio de producción (variable `WEB_URL`).
GitHub	Secrets, Variables y los ambientes `preview`, `api-production` y `web-production`.	—

Límites que conviene conocer

- **Vercel (plan Hobby)** está pensado para proyectos personales y no comerciales: es adecuado para esta práctica, no para un cliente. Revisa los términos vigentes antes de usarlo en un proyecto real.
- **Render (plan Free)** duerme el servicio tras unos 15 minutos sin tráfico y tarda cerca de un minuto en despertar; además limita las horas mensuales. Es la causa de las esperas y los reintentos del smoke test.
- **GitHub Actions** no cobra minutos en repositorios públicos; en los privados hay una cuota mensual. Un workflow sin `concurrency` puede gastarla con runs duplicados.
- Los menús y nombres de las plataformas cambian con frecuencia. Si algo no coincide con la guía, busca el concepto (Root Directory, Deploy Hook, Environment Variables, Deployment Protection) y no el recorrido de clics.

Reglas de seguridad del pipeline

- Un secreto vive en Settings, nunca en un archivo del repositorio. Si se filtra, se **revoca**; borrar el commit no alcanza porque el historial lo conserva.
- GitHub enmascara el valor exacto de un secreto en los logs. Si lo transformas (base64, partes), deja de enmascararse: no lo imprimas.
- Los pull requests que vienen de un fork no reciben secretos. Es una protección, no un fallo: un colaborador externo no debe poder ejecutar código con tus credenciales.
- `permissions` se declara en cada workflow con lo mínimo (`contents: read`) y solo se amplía en el job que lo necesita (`pull-requests: write` en el preview).
- Los tokens llevan alcance mínimo y vencimiento. El de Vercel debe caducar al terminar la práctica.
- Las actions de terceros se fijan a una versión (`@v4`), no a `@main`: una versión nueva no debe cambiar tu pipeline sin que lo notes.
- Los datos que el pipeline imprime (URLs de despliegue, el SHA) no son secretos; el Deploy Hook de Render **sí lo es**, porque cualquiera con esa URL puede pedir un despliegue.